

Team Project

Rules & Process

A few days. One team. One presentation.



The Presentation

- ▶ **15 minutes** to present your project, **5 minutes of Q&A**
- ▶ Presented in front of **some of your managers** — this is real, not a rehearsal
- ▶ You will need: a **PowerPoint (or equivalent) slide deck** and a **live demo**
- ▶ You can also show anything else that reveals your process: git graph, board, timeline, test coverage report — anything that tells the story of how you built it
- ▶ Managers will ask questions. Be ready to explain and defend your decisions, not just show features



Designing Your Slides

- ▶ **Light, not heavy** — one key idea per slide, not a wall of text
- ▶ Slides **support** you — they are not a script to read from word-for-word
- ▶ Bullet points, not paragraphs — a few words, not full sentences
- ▶ Prefer a diagram, screenshot, or number over a dense paragraph of text
- ▶ If the audience is reading your slide, they' ve stopped listening to you

Your Instructors Play the Customer

- ▶ During the project, instructors act as your **customer** — treat them like a stakeholder, not a teacher with all the answers
- ▶ Ask them clarifying questions about scope and requirements, the way you would a real client
- ▶ They will step in to help only for **major issues** — day-to-day technical struggles are yours to work through as a team
- ▶ This mirrors a real client relationship: you don't get unlimited hand-holding, you get direction when it truly matters



Part 1 · Your Repository Is Your Shop Window

- ▶ Your Git repository is your **vitrine** — it's what managers and instructors will actually look at
- ▶ A clean, active, well-documented repository tells a better story than any slide can
- ▶ History matters as much as the final code: how you got there is the whole point of this exercise
- ▶ You should always be able to explain what's in your repository, and why
- ▶ Everything in this section is about keeping that shop window worth looking at



Branching Strategy

Branch	Purpose
<code>main</code> / <code>prod</code>	What you'd ship — always in a working state
<code>qa</code> / <code>release</code>	Integration branch — where features come together before prod
<code>feature/*</code>	One branch per feature or task — merged in via Pull Request

Commit Discipline & Messages

- ▶ Commit **regularly** — small, meaningful commits, not one giant commit at the end of the day
- ▶ As a **team**, avoid a big gap between how much each person has committed
- ▶ A repo where one person has 80 commits and another has 3 tells its own (bad) story

✗ "fix" • "wip" • "update stuff"

✓ "Fix null pointer when account has no transactions yet"

✓ "Add validation for duplicate account numbers"

Explain **why**, not just what — the diff already shows what changed.

Pull Requests & Code Review

- ▶ **Pull Requests are mandatory** — no direct commits to `main / qa`
- ▶ Every PR needs **code review by a human teammate** — not by AI, and not a rubber-stamp approval
- ▶ Reviewers should actually read the diff and ask questions if something is unclear
- ▶ This is also how the whole team stays aware of what's being built, not just their own slice

Unit Testing Discipline

- ▶ **Backend unit tests are mandatory** — not optional, not "if we have time"
- ▶ **Minimum 70% line coverage** on the backend
- ▶ Tests should validate **business logic**, not just the happy path — cover edge cases, invalid input, failure modes
- ▶ A test suite that only checks the easy case doesn't prove much — and it won't impress anyone reviewing your repo

Document Everything

- ▶ Every meaningful **decision**, **mistake**, and **choice** should be explained and documented somewhere
- ▶ README, ADRs (architecture decision records), commit messages, PR descriptions — pick what fits, but write it down
- ▶ "We chose X over Y because..." is worth ten lines of undocumented code
- ▶ If you can't remember why you did something a week later, neither will the person reading your repo

Frontend Approach & Tools

- ▶ **Plain JavaScript, no framework at all, is completely fine**

- ▶ Or use **any framework**, as long as **2+ teammates** already know it

- ▶ Nobody qualifies? Pick together and teach (**3-4h+**) — document that too

- ▶ Either way, document your reasoning

- ▶ Use whatever tools help you: Jira, Trello, Notion, a spreadsheet

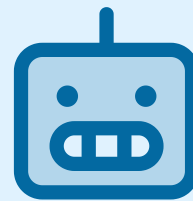
- ▶ Bring **graphs and visuals**: git graph, timeline, workload distribution

- ▶ Anything that showcases your process visually is worth including



Part 2 · Human-Driven Code Structure

- ▶ You can — and will — use AI. But with discipline.
- ▶ **Developers decide**, not the AI: package structure, class responsibilities, method names
- ▶ Copilot (or any assistant) suggestions can be **refined** — they should not be **blindly accepted**
- ▶ If a suggestion doesn't fit how your team structures things, change it



Understand What Is Generated

- ▶ Instructors will come by **during the project**, at random — not just on presentation day — and ask you to open a piece of code and explain it
- ▶ If you are not able to explain it: **we will ask you to remove it from your repository**
- ▶ This isn't a gotcha — it's the whole point. Code you don't understand is a liability, not progress
- ▶ Read what the AI gives you before you accept it, every time
- ▶ Treat every day like it could be checked — not out of fear, but because that's how real teams work

Acceptance Criteria Validation

- ▶ Code must be **manually checked** against the acceptance criteria by the developer — not assumed correct because it compiled
- ▶ **"Works on my machine" is not completion**
- ▶ **AI output ≠ Done** — generated code is a draft, not a finished, verified feature
- ▶ You decide when something is actually done, based on the criteria — not the assistant

Debug Like a Human First

- ▶ **First attempt:** logs, breakpoints, reasoning through the problem yourself
- ▶ Only reach for Copilot **after** you understand the issue, not as the first reflex
- ▶ **AI is a second brain, not the first reaction**
- ▶ Debugging is where you actually learn the codebase — don't skip that by outsourcing it immediately



The Explain-Your-Code Rule

Any Copilot-generated code must be explainable by the developer — **line by line**.

If you can't explain it:



Rewrite it — or —  **Don't merge it**



Part 3 · Plan & Communicate

- ▶ **Kickoff & planning → Build → Integrate & stabilise → Polish & rehearse**

- ▶ Leave real time for the last step — rehearsed beats improvised

- ▶ Don't jump straight into code on day one

- ▶ A short daily sync helps more than it costs

- ▶ Surface blockers early — don't sit on them alone

- ▶ Help a stuck teammate rather than pushing ahead solo

Prepare for Demo Day

- ▶ **Dry run: Wednesday or Thursday** — a practice presentation where you get real feedback before the actual day
- ▶ **Rehearse the live demo** — more than once, as a team, out loud
- ▶ Have a **backup plan**: a recording or screenshots in case the live demo fails
- ▶ Assign speaking parts so everyone is visibly involved

Before you present, check:

- ▶  Tests cover more than the happy path
- ▶  Every PR reviewed by a teammate
- ▶  Commits are regular & balanced
- ▶  You can explain every line, live
- ▶  Decisions & mistakes are documented

Tell the Story, Mistakes Included

- ▶ Don't just list features — tell the **story** of how you built this, decisions and all
- ▶ You **will** make mistakes. That's expected — this is how real projects work
- ▶ "We made mistake X, and fixed it using Y and Z" is a **good story**, not a confession
- ▶ Teams that hide mistakes look worse than teams that explain how they recovered from them





One More Time

**We are not judging the result.
We are judging the **process**.**

Build something real, make mistakes, document what happened, and be ready to explain every line of it. That's the whole assignment.

And it's **not a race** — there's no prize for finishing first or cramming in the most features.



What's Next · Set Up Your Repository

- ▶ **Choose a team name** — first, as a team
- ▶ **Create your Git repository** — will be **PRIVATE** for now
- ▶ **Set Owner to your GitHub group** — not personal, see screenshot →
- ▶ **Add teammates** as collaborators, then **push a README**
- ▶ **Everyone clones the repo** before you start building

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner *
Neueda-Learning

Repository name *

⚠ Name cannot be blank

Great repository names are short and memorable. How about [solid-parakeet?](#)

Description

Set **Owner** to the GitHub group you were added to — not your personal account.

Register Your Project

- ▶ At the **end of the day**, you' ll be given a **QR code**
- ▶ **Scan it** to officially register your project — using the **team name** you chose earlier
- ▶ This is how we know your team and repository exist for the presentation — don' t skip it

